



FULL-STACK APPLICATION DESIGN AND DEVELOPMENT

COURSE 13 - PROJECT MANAGEMENT



WHAT IS PROJECT MANAGEMENT?

- Project management involves the strategic planning, development, and delivery of products. Various methodologies have emerged to streamline and optimize this process.
- There are four popular frameworks:
 - Waterfall
 - Agile
 - Scrum
 - Kanban.

Waterfall



Agile



Scrum



Kanban

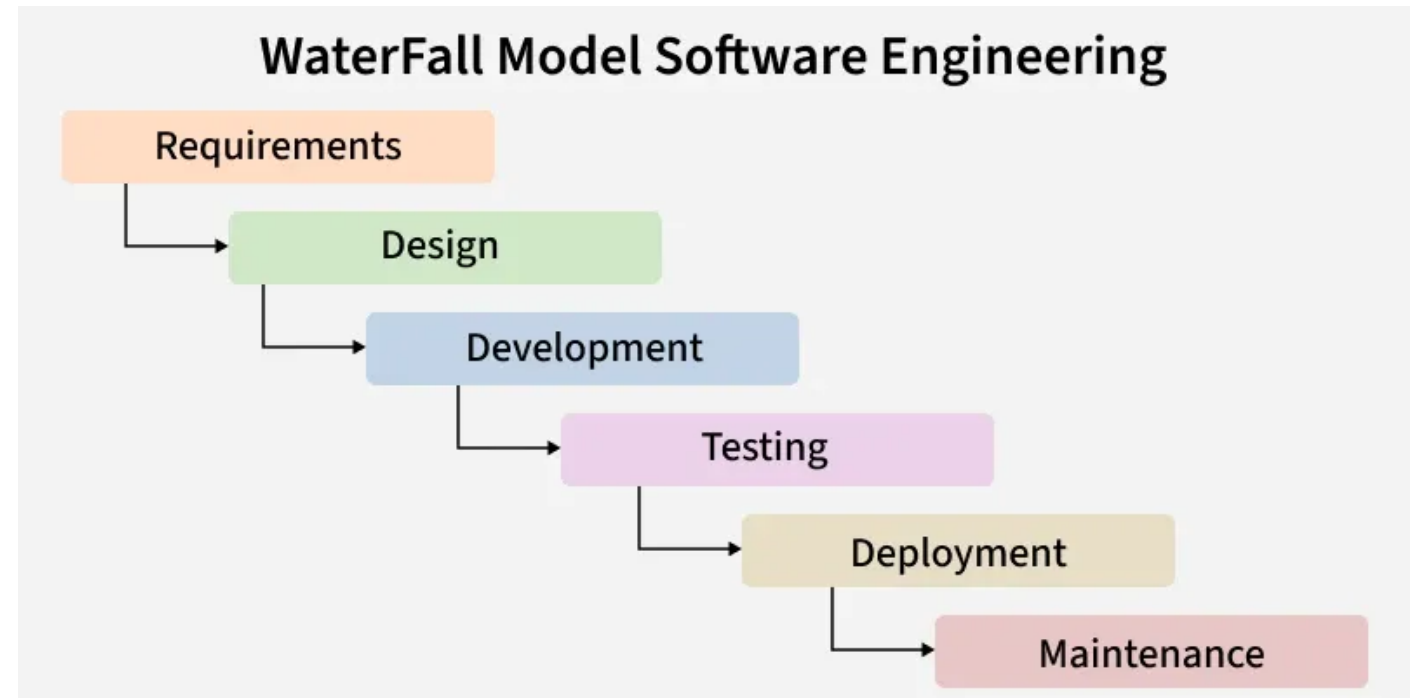


WATERFALL METHODOLOGY

- The waterfall model is a Software Development Model used in the context of large, complex projects, typically in the field of information technology. It is characterized by a structured, sequential approach to **Project Management** and **Software Development**.
- The Waterfall Model is useful in situations where the project requirements are well-defined and the project goals are clear. It is often used for large-scale projects with long timelines, where there is little room for error and the project stakeholders need to have a high level of confidence in the outcome.

WATERFALL METHODOLOGY

- **Phases of Waterfall Model**
- **Classical Waterfall Model** divides the life cycle into a set of phases. The development process can be considered as a sequential flow in the waterfall. The different sequential phases of the classical waterfall model are as follows:



WATERFALL METHODOLOGY

I. Requirement Analysis and Specification

- **Requirement Analysis and Specification** phase aims to understand the exact requirements of the customer and document them properly. This phase consists of two different activities.
 - A. **Requirement Gathering and Analysis:** Firstly all the requirements regarding the software are gathered from the customer and then the gathered requirements are analyzed.
 - The goal of the analysis part is to remove incompleteness (an incomplete requirement is one in which some parts of the actual requirements have been omitted) and inconsistencies (an inconsistent requirement is one in which some part of the requirement contradicts some other part).
- B. **Requirement Specification:** These analyzed requirements are documented in a software requirement specification (SRS) document. SRS document serves as a contract between the development team and customers. Any future dispute between the customers and the developers can be settled by examining the SRS document.

WATERFALL METHODOLOGY

2. Design

- The goal of the **Software Design Phase** is to convert the requirements acquired in the SRS into a format that can be coded in a programming language. It includes high-level and detailed design as well as the overall software architecture. A **Software Design Document** is used to document all of this effort (SDD).
 - **High-Level Design (HLD):** This phase focuses on outlining the broad structure of the system. It highlights the key components and how they interact with each other, giving a clear overview of the system's architecture.
 - **Low-Level Design (LLD):** Once the high-level design is in place, this phase zooms into the details. It breaks down each component into smaller parts and provides specifics about how each part will function, guiding the actual coding process.

WATERFALL METHODOLOGY

3. Development

- In the **Development Phase** software design is translated into source code using any suitable programming language. Thus each designed module is coded. The unit testing phase aims to check whether each module is working properly or not.
 - In this phase, developers begin writing the actual source code based on the designs created earlier.
 - The goal is to transform the design into working code using the most suitable programming languages.
 - Unit tests are often performed during this phase to make sure that each component functions correctly on its own.

WATERFALL METHODOLOGY

4. Testing and Deployment

- A. Testing:** Integration of different modules is undertaken soon after they have been coded and unit tested. Integration of various modules is carried out incrementally over several steps. During each integration step, previously planned modules are added to the partially integrated system and the resulting system is tested. Finally, after all the modules have been successfully integrated and tested, the full working system is obtained and system testing is carried out on it. System testing consists of three different testing activities:
 - **Alpha testing:** Alpha testing is the system testing performed by the development team.
 - **Beta testing:** Beta testing is the system testing performed by a friendly set of customers.
 - **Acceptance testing:** After the software has been delivered, the customer performs acceptance testing to determine whether to accept the delivered software or reject it.
- B. Deployment:** Once the software has been thoroughly tested, it's time to deploy it to the customers or end-users. This means making the software ready and available for use, often by moving it to a live or staging environment.
 - During this phase, we also focus on helping users get comfortable with the software by providing training, setting up necessary environments, and ensuring everything is running smoothly. The goal is to make sure the system works as expected in real-world conditions and that users can start using it without any problems.

WATERFALL METHODOLOGY

5. Maintenance

- The **Maintenance Phase** is the most important phase of a software life cycle. The effort spent on maintenance is 60% of the total effort spent to develop a full software. There are three types of maintenance:
 - **Corrective Maintenance:** This type of maintenance is carried out to correct errors that were not discovered during the product development phase.
 - **Perfective Maintenance:** This type of maintenance is carried out to enhance the functionalities of the system based on the customer's request.
 - **Adaptive Maintenance:** Adaptive maintenance is usually required for porting the software to work in a new environment such as working on a new computer platform or with a new operating system.

WATERFALL METHODOLOGY

■ Advantages of Waterfall Model

- **Easy to Understand:** The model is very simple and easy to understand.
- **Individual Processing:** Phases in the model are processed one at a time.
- **Properly Defined:** Each stage in the model is clearly defined.
- **Clear Milestones:** The model has very clear and well-understood milestones.
- **Properly Documented:** Processes, actions, and results are very well documented.
- **Reinforces Good Habits:** The model reinforces good habits like define-before-design and design-before-code.
- The Waterfall Model works well for smaller projects and projects where requirements are well understood.

WATERFALL METHODOLOGY

■ Disadvantages of Waterfall Model

- **No Feedback Path:** In the Waterfall Model evolution of software from one phase to another phase is like a waterfall. It assumes that no error is ever committed by developers during any phase. Therefore, it does not incorporate any mechanism for error correction.
- **Difficult to Accommodate Change Requests:** This model assumes that all the customer requirements can be completely and correctly defined at the beginning of the project, but the customer's requirements keep changing with time. It is difficult to accommodate any change requests after the requirements specification phase is complete.
- **No Overlapping of Phases:** This model recommends that a new phase can start only after the completion of the previous phase. But in real projects, this can't be maintained. To increase efficiency and reduce cost, phases may overlap.
- **Limited Flexibility:** The Waterfall Model is a rigid and linear approach to software development, which means that it is not well-suited for projects with changing or uncertain requirements. Once a phase has been completed, it is difficult to make changes or go back to a previous phase.
- **Limited Stakeholder Involvement:** The Waterfall Model is a structured and sequential approach, which means that stakeholders are typically involved in the early phases of the project (requirements gathering and analysis) but may not be involved in the later phases (implementation, testing, and deployment).
- **Late Defect Detection:** In the Waterfall Model, testing is typically done toward the end of the development process. This means that defects may not be discovered until late in the development process, which can be expensive and time-consuming to fix.
- **Lengthy Development Cycle:** The Waterfall Model can result in a lengthy development cycle, as each phase must be completed before moving on to the next. This can result in delays and increased costs if requirements change or new issues arise.

WATERFALL METHODOLOGY

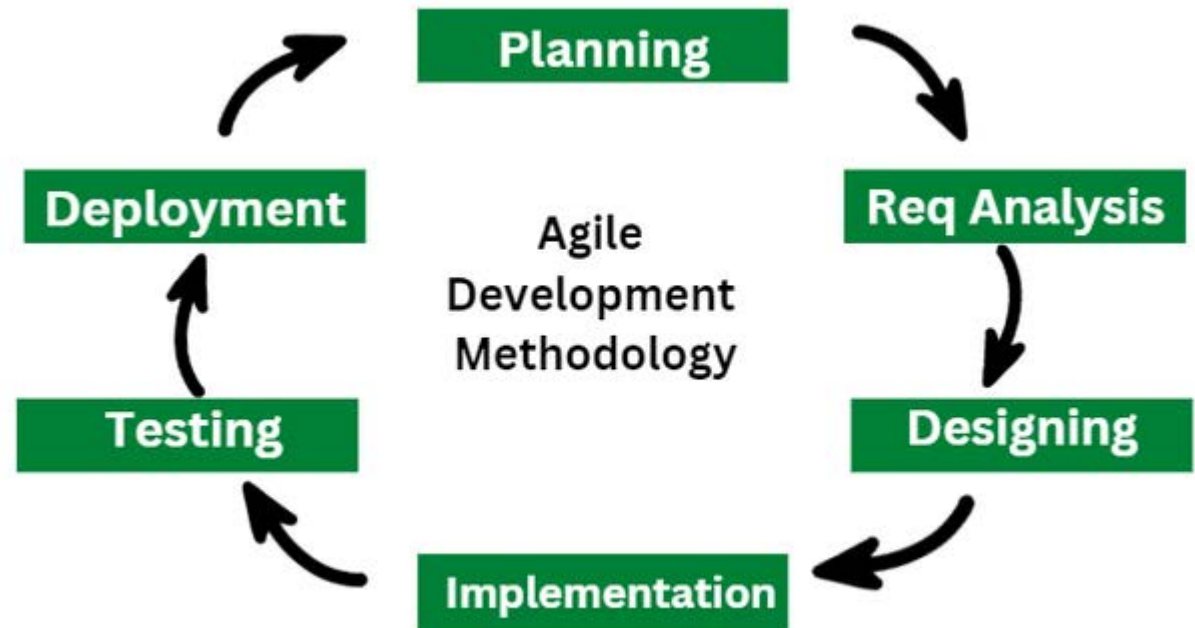
- **When to Use Waterfall Model?**
- Here are some cases where the use of the Waterfall Model is best suited:
 - **Well-understood Requirements:** Before beginning development, there are precise, reliable, and thoroughly documented requirements available.
 - **Very Little Changes Expected:** During development, very little adjustments or expansions to the project's scope are anticipated.
 - **Small to Medium-Sized Projects:** Ideal for more manageable projects with a clear development path and little complexity.
 - **Predictable:** Projects that are predictable, low-risk, and able to be addressed early in the development life cycle are those that have known, controllable risks.
 - **Regulatory Compliance is Critical:** Circumstances in which paperwork is of utmost importance and stringent regulatory compliance is required.
 - **Client Prefers a Linear and Sequential Approach:** This situation describes the client's preference for a linear and sequential approach to project development.
 - **Limited Resources:** Projects with limited resources can benefit from a set-up strategy, which enables targeted resource allocation.
- The Waterfall approach involves less user interaction in the product development process. The product can only be shown to end users when it is ready.

AGILE METHODOLOGY

- **Agile Methodology** is a **way to manage projects by breaking them into smaller parts**. It focuses on working together and making constant improvements. Teams plan, work on the project, and then review how things are going in a repeating cycle.
 - They prioritize flexibility, collaboration, and customer satisfaction.
 - Major companies like Facebook, Google, and Amazon use Agile because of its adaptability and customer-focused approach.

AGILE METHODOLOGY

- Life cycle of Agile Methodology
- The **Agile Software Development Life Cycle** helps you break down each project into six simple stages:



AGILE METHODOLOGY

1. Requirement Gathering

- In this stage, the project team identifies and documents the needs and expectations of various stakeholders, including clients, users, and subject matter experts.
- It involves defining the **Project's Scope**, objectives, and requirements.
- Establishing a budget and schedule.
- Creating a project plan and allocating resources.

2. Design

- Developing a high-level system architecture.
- Creating detailed specifications, which include data structures, algorithms, and interfaces.
- Planning for the software's user interface.

AGILE METHODOLOGY

3. Development (Coding)

- Writing the actual code for the software.
- Conducting unit testing to verify the functionality of individual components.

4. Testing

- This phase involves several types of testing:
 - **Integration Testing:** Ensuring that different components work together.
 - **System Testing:** Testing the entire system as a whole.
 - **User Acceptance Testing:** Confirming that the software meets user requirements.
 - **Performance Testing:** Assessing the system's speed, scalability, and stability.

AGILE METHODOLOGY

5. Deployment

- Deploying the software to a production environment.
- Placing the software into the real world where people can use it.
- Making sure it works smoothly in the real world.
- Providing training and support for end-users.

6. Review (Maintenance)

- Addressing and resolving any issues that may arise after deployment.
- Releasing updates and patches to enhance the software and address problems.

AGILE METHODOLOGY

■ Advantages of Agile Model

- **Flexibility and Adaptability:** Agile can quickly adapt to changes, allowing teams to respond to new customer needs and market conditions.
- **Improved Collaboration:** Agile encourages constant communication between developers and stakeholders, ensuring the product meets user expectations.
- **Faster Delivery:** Agile ensures quicker releases, keeping customers engaged and their feedback incorporated early.
- **Enhanced Quality and Customer Satisfaction:** Agile focuses on customer feedback, ensuring the product meets their needs and delivering high-quality results.
- **Iterative Development:** Work is done in small, manageable steps, allowing for regular improvements and quick adjustments.
- **Transparency:** Agile keeps stakeholders informed at every stage, ensuring clarity and alignment.
- **Quality Assurance:** Agile prioritizes quality, ensuring the product meets users' expectations through continuous improvements.
- **Continuous Improvement:** Regular feedback ensures the product keeps improving, preventing last-minute issues and maintaining high quality.

AGILE METHODOLOGY

■ Disadvantages of Agile Model

- **Less Documentation:** Agile methodologies focus on less documentation; it prioritizes working on projects rather than paperwork.
- **Challenges in Large Organizations:** Busy schedule of clients can make daily meetup and face-to-face communication difficult.
- **Need for Senior Programmers:** It may require experienced programmers to make critical decisions during the development of software.
- **Limited Scope Control:** It has less rigid scope control, which may not be suitable in certain situations.
- **Predictability:** Compared to more structured project management methods, it may lack predictability.

AGILE METHODOLOGY

- **When to use the Agile Methodology?**
- **The Agile Methodology** is particularly well-suited for projects and organizations where the following conditions or needs are present:
 - **Unclear or Changing Requirements:** Agile is great for projects with requirements that aren't well-defined or might change.
 - **Complex Projects:** It's good for big, complex projects by breaking them into smaller pieces.
 - **Customer Focus:** Use Agile when making customers happy is a priority and you want to involve them throughout.
 - **Quick Time-to-Market:** If you need to get your product out fast, Agile can help.
 - **Small to Medium Teams:** Agile works well for teams of a few to a few dozen people.
 - **Team Skills:** It's best when you have a mix of skills in your team, like development, testing, design, and more.
 - **Collaboration:** Agile promotes working together and open communication.
 - **Regular Updates:** If you want to check progress often and make changes as needed.
 - **Transparency:** Agile emphasizes being open and clear with everyone involved in the project.
 - **Risk Control:** It helps manage risks by tackling issues as they come up.
 - **Continuous Improvement:** Agile fosters a culture of always getting better over time.

SCRUM METHODOLOGY

- Scrum is a widely used project management framework in the rapidly changing field of Agile development. In Agile project management, a Scrum Team is essential for creating high-quality products quickly.
- **Scrum Team**
- A Scrum Team is a group that works on projects using a method called Scrum. In this group, different people can do various jobs, and they work together to create a part of a product. They aim to finish and deliver something valuable in a short period, which they call a "**Sprint**".



SCRUM METHODOLOGY

- **Team Size:** Scrum teams are small, usually between five to nine members. It's better to have multiple small teams working on a feature rather than larger teams.
- **Cross-Functional Collaboration:** Scrum teams are designed to deliver complete features, emphasizing the importance of collaboration among team members. Cross-functional teams support each other's strengths and weaknesses.
- **Self-Organization:** Teams are set up to be self-organizing, which means they have a clear purpose and are encouraged to work independently.
- **Input into Team Design:** Ideally, team members should be involved in shaping how their teams are organized.

SCRUM METHODOLOGY

- **Scrum Team Roles and Responsibilities**

- **Product Owner**

- The team member who understands the customer's needs and their business value is known as the **Product Owner**. This person communicates the customer's requirements and priorities to the Scrum team. The Product Owner must understand both the product's business case and the features customers want. To ensure the team is building the right product and following the product vision, the Product Owner must be readily available for consultation.
- The Product Owner is responsible for managing the **Product Backlog** and must have the authority to make decisions necessary to move the project forward. Their responsibilities include:
 - Clearly defining and describing Product Backlog items.
 - Prioritizing Product Backlog items to best achieve project goals and objectives.
 - Maximizing the value of the work completed by the development team.
 - Ensuring the Product Backlog is transparent, accessible, and understood by all stakeholders, so everyone knows what the team will work on next.

SCRUM METHODOLOGY

- **Scrum Master**
- The scrum master assists in removing any obstacles that might be affecting the team's productivity and in holding members of the team accountable for their commitments to the company. They review work and deliverables with the team regularly (usually once a week). A scrum master's job is to coach and inspire team members, not impose rules on them.
- A scrum master's responsibilities include:
 - Making sure everything goes as planned.
 - Removing any barriers that affect output.
 - Planning the important meetings and events.

SCRUM METHODOLOGY

- **Development Team**
- The organization provides structure and authority to Development Teams so they can plan and manage their own work. This autonomy increases the team's energy, efficiency, and overall effectiveness. Development Teams share the following characteristics:
 - They are self-managing and are not told how to turn Product Backlog items into potentially releasable increments of functionality, not even by the Scrum Master.
 - They are cross-functional, meaning the team has all the skills necessary to deliver a product increment.
 - Scrum does not recognize sub-teams within the Development Team, regardless of specialized areas such as testing, architecture, operations, or business analysis.
 - While individual team members may have specialized skills or focus areas, the Development Team as a whole is collectively responsible for the work and its outcomes.

SCRUM METHODOLOGY

- **What is the Ideal Size of a Scrum Team?**
- Scrum teams are typically composed of five to nine members, although seven is the optimal number. The development team consists of five to seven people, plus one scrum master and a product owner. Sub-teams do not exist. The scrum team members should be full-time employees, preferably based in the same office. If the work needs to be done in multiple locations, a scrum team should be assigned to each location.

SCRUM METHODOLOGY

■ Benefits of a Scrum Team

- **Work happens simultaneously:** Instead of completing tasks one after another, Scrum teams work on multiple components at the same time. This saves time, allows for continuous improvements throughout the project, and encourages collaboration and diverse perspectives, leading to higher-quality outcomes.
- **Clear workflow processes:** Scrum provides well-defined phases (such as planning, sprints, daily scrums, reviews, and retrospectives) that help teams stay focused and organized. Each phase supports collaboration, with sprints serving as short development cycles focused on delivering shippable products.
- **Higher ROI and reduced risk:** Organizations using Scrum often experience increased return on investment because teams work faster, make fewer costly mistakes, and use resources more efficiently. Consistently high ROI also lowers overall project management risk.
- **Improved team morale:** Scrum and Agile principles emphasize people, communication, and teamwork. Regular feedback, reflection, and sustainable work practices help teams adapt, stay engaged, and continuously improve.

KANBAN METHODOLOGY

- Kanban is a popular Agile Software Development Methodology. It is a signaling device that instructs the moving of parts in a 'pull' production system, developed as part of the TPS (Toyota Production System). Kanban is about envisioning the existing workflow in terms of steps. These steps can be created on the whiteboard.
- The main aim of Kanban is to reduce WIP (Work-In-Progress), or inventory, between processes by ensuring the upstream process creates parts as long as its downstream process needs it. The goal of the Kanban execution is to ensure work items move to the next steps quickly to realize business value faster.

KANBAN METHODOLOGY

- **Kanban Practices**

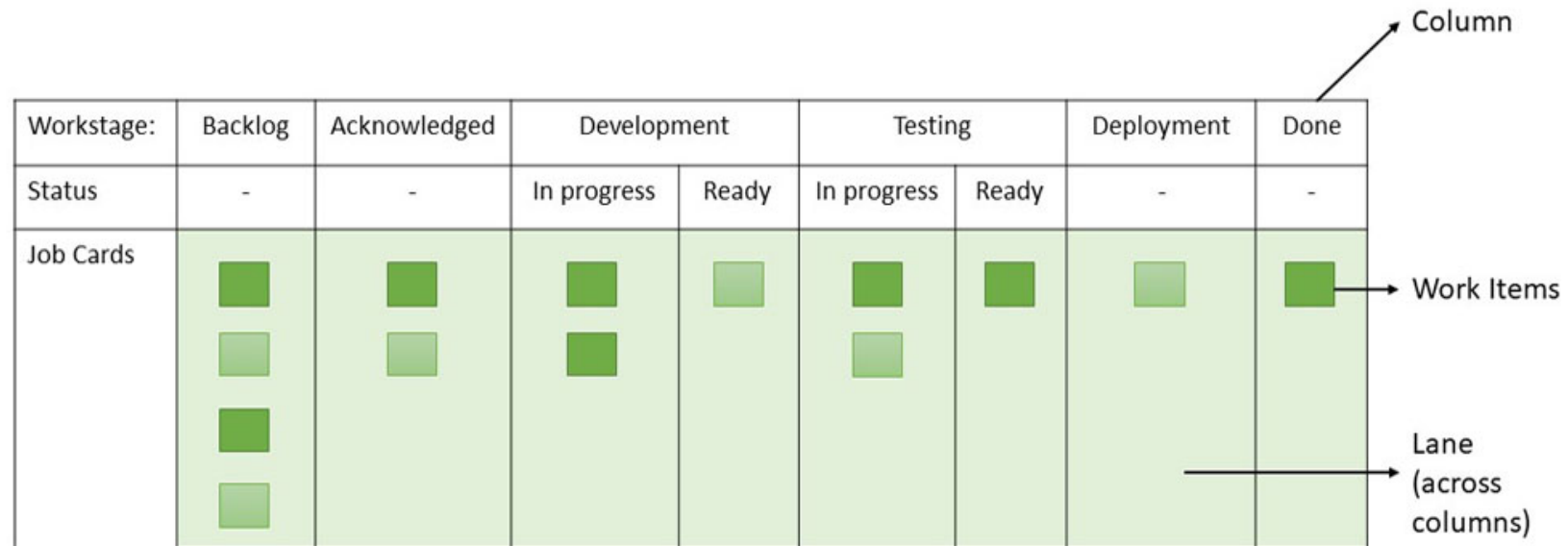
- The following are the six core Kanban practices:

- **Limit WIP:** Limits for Work-In-Process control how many items can be worked on at each stage. This pull-based system helps reduce bottlenecks and encourages continuous, incremental improvement.
- **Visualize work:** Kanban makes work visible, usually through a board with columns and cards that represent workflow stages. This visibility helps teams understand how work moves through the system.
- **Manage flow:** Teams monitor and measure how work flows through each stage to identify issues and evaluate whether changes improve or hinder performance.
- **Improve collaboratively, evolve experimentally:** Kanban promotes small, continuous improvements. Shared understanding of work and risks helps teams collaborate effectively and agree on improvement actions.
- **Implement feedback loops:** Regular feedback from customers, stakeholders, and processes helps reduce risk and optimize delivery.
- **Make policies explicit:** Clearly defined rules and processes allow teams to have objective, data-driven discussions and reach consensus on improvements.

KANBAN METHODOLOGY

- **Kanban Board/Card**
- Understanding how workflow stages are visualized in the task execution pipeline is critical. A Kanban board provides a simple and effective way to visualize and manage this process. It works as follows:
 - Every request, bug, or ticket is placed on the Kanban board.
 - Each column on the board represents a **work stage** in the lifecycle of a ticket or bug. For example, a board might include the stages *Received/Acknowledged*, *In Progress*, *UAT (User Acceptance Testing)*, and *Done*.
 - The *Received* stage may also be referred to as the **Backlog**.
 - Teams can define and name workflow stages based on their own processes and terminology.
 - A Kanban board can be a physical whiteboard using sticky notes or an electronic Kanban board.
 - Application Lifecycle Management (ALM) tools such as Rally or Jira can be configured to support Kanban boards.
 - The board helps identify bottlenecks by signaling when tickets remain in a stage for too long.
 - On electronic boards, tickets or user stories can display timestamps automatically.
 - On physical boards, teams can manually record the date and time on each ticket to track progress.

KANBAN METHODOLOGY



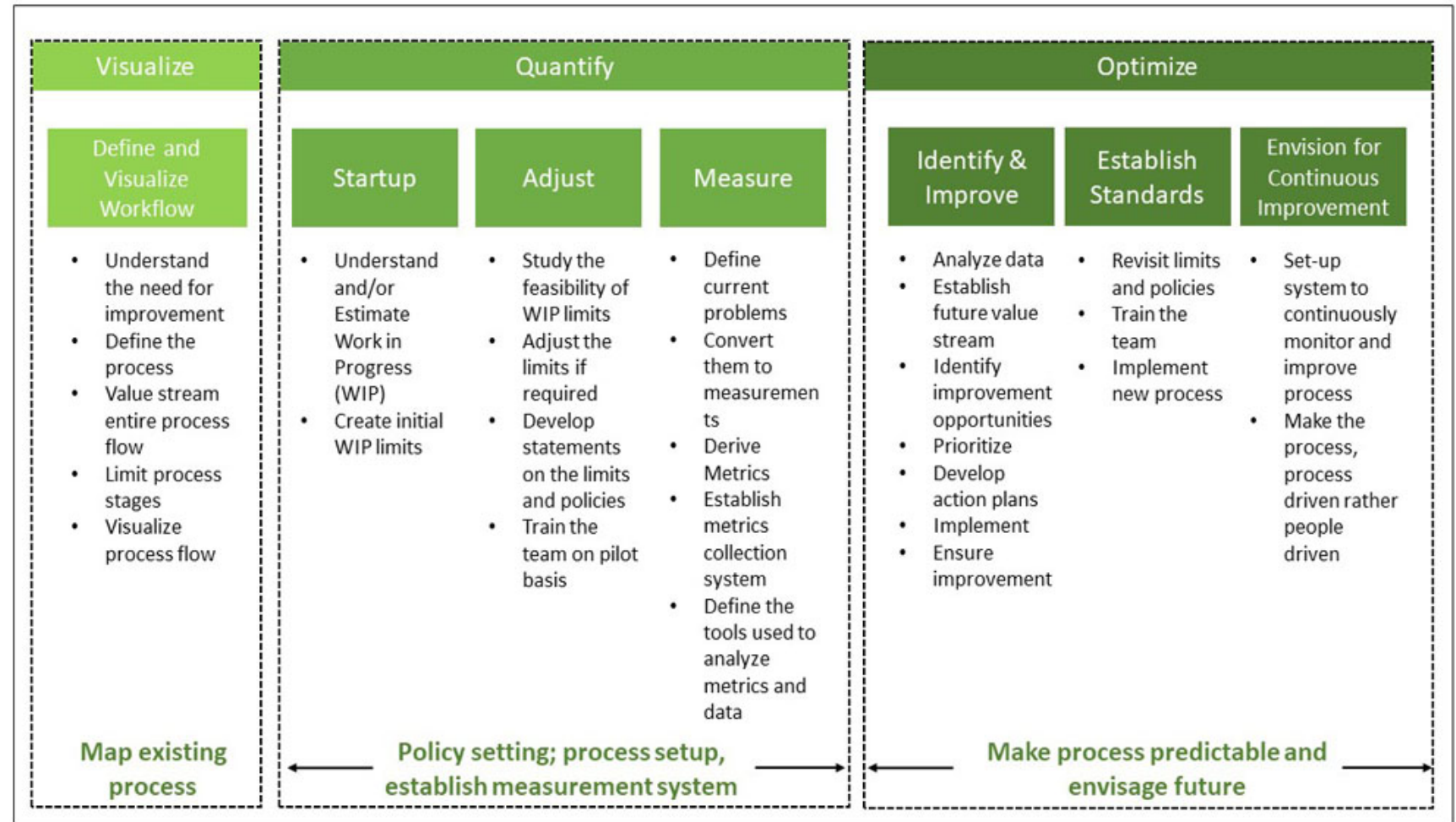
 Work Item (Type 1)

 Work Item (Type 2)

KANBAN METHODOLOGY

- The Kanban approach involves three steps:

- Visualize
- Quantify
- Optimize



KANBAN METHODOLOGY

- Kanban is best suited for the next scenarios:
 - Environments with dynamic or frequently changing requirements that need to be delivered quickly.
 - Situations where priorities change often and the team can pull the highest-priority work as soon as WIP limits allow.
 - Projects that require frequent or periodic releases.
 - Work environments where incoming tasks are continuous.
 - Cases where task priority must be determined dynamically based on the nature and type of work.
 - Ticket-based work or production support projects.
- However, Kanban may not be the best fit in the following situations:
 - Tasks frequently remain in a “waiting” state for long periods.
 - Work is primarily research-oriented.
 - Enhancement projects where requirements are unclear or constantly evolving.
 - Projects with no defined scope, where tasks continuously change or expand.
 - Work with heavy dependencies between tasks.
 - Scenarios where all work items across stages must be completed and combined before deployment.

CHOOSING THE RIGHT METHODOLOGY

Aspect	Waterfall	Agile	Scrum	Kanban
Approach	Sequential	Iterative and Incremental	Iterative within Agile framework	Visual and Continuous Delivery
Flexibility	Low	High	Moderate	High
Phases	Distinct phases (linear)	Iterative cycles (requirements, design, implementation, testing)	Fixed-duration sprints (2-4 weeks)	Visual workflow, continuous delivery
Adaptability	Low (Changes are challenging)	High (Welcomes changes throughout development)	High (Adaptable to changing requirements)	High (Emphasizes continuous improvement)
Customer Collaboration	Limited	High	High	Moderate (Focus on workflow optimization)
Roles	Fixed roles (e.g., project manager, developer)	Collaborative roles (cross-functional teams)	Defined roles (Product Owner, Scrum Master, Development Team)	Flexible roles, often cross-functional
Delivery Frequency	At the end of the project	Incremental, regular releases	At the end of each sprint	Continuous, as tasks are completed
Visual Management	Limited	Limited	Scrum board for sprint progress	Visual boards for workflow management
Work in Progress (WIP)	Not applicable	Not explicitly defined	Limited by sprint capacity	Limited by team's capacity and goals
Best Suited For	Well-defined projects with stable requirements	Projects with evolving or unclear requirements	Complex projects with dynamic requirements	Projects with a continuous flow of work

CHOOSING THE RIGHT METHODOLOGY

■ Summary

- **Waterfall:** Best for well-defined projects with stable requirements and smaller teams.
- **Agile:** Suited for projects with evolving requirements and a need for continuous adaptation.
- **Scrum:** Ideal for complex projects with dynamic requirements and a focus on collaboration.
- **Kanban:** Suitable for projects with a continuous flow of work and teams that benefit from visual management.